DS605 · Fundamentals of Machine Learning

Lecture Notes 15

Linear Regression: Predicting Order Value on ShopKart

Today's question, in one sentence: several straight lines could explain a pattern in data — how does an algorithm decide which one is best, and what does "fit" mean, mathematically? By the end of this lecture, you will be able to derive, not just apply, the formula scikit-learn uses.

Roadmap

NOTE · Act I — The Problem of Choice

Residuals, squared error, the loss function.

CAUTION · Act II — Many Dimensions

Vector notation, planes, and hyperplanes.

NOTE · Act III — Solving It

The normal equation, gradient descent.

WORKED EXAMPLE · Act IV — When Lines Aren't Enough

Polynomial regression, and the overfitting cliffhanger that opens Lecture 16.

Act I — The Problem of Choice

Opening Problem

Suppose you are told how many items are in a customer's cart on ShopKart. Can you predict the order's total value? And if several straight-line models could all describe the pattern reasonably well, how should an algorithm decide which one is best?

The ShopKart Order Dataset

ShopKart is DS605's running example: a simulated e-commerce marketplace. We use a simulated dataset of m = 4,000 orders to learn a linear regression model.

OrderID	Items	Distance (km)	Order Value	Weight (kg)	City Tier
SK20245457	9	0.7	₹1,189.43	9.3	Tier 1
SK20247049	3	12.0	₹474.62	5.0	Tier 1
SK20246776	19	2.4	₹2,785.43	5.9	Tier 3
SK20249551	12	8.7	₹1,784.66	3.0	Tier 2

(4 of 4,000 simulated rows shown, with membership and payment columns omitted for space. The dataset is generated for teaching purposes, not scraped from a real platform.) Today, we model Order Value using only Items in Cart. Later, we bring in more columns.

A scatter plot of Items in Cart against Order Value shows a pattern that a straight line could plausibly describe — but many different lines could be drawn through the same cloud of points. Which one should the algorithm choose, and why?

The Prediction Function

For a single feature (Items in Cart), a line predicts. This is our model, or hypothesis:

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

This is the prediction $\hat{y}_i$ — not the observed value y_i. They are different symbols for a reason: one is what the model outputs, the other is what actually happened.

Meaning of β_0 and β_1

WORKED EXAMPLE · β_1 — slope

The change in predicted order value for one additional item in the cart.

THINK ABOUT IT · β_0 — intercept

The predicted order value when the cart hypothetically has zero items — a mathematical anchor, not necessarily a physically meaningful quantity.

β_0 at Items = 0 is an extrapolation outside the data's natural range and shouldn't be over-interpreted — a first taste of “don't extrapolate blindly.”

Residuals

For each order i, define the residual as how far off the line's prediction was for that order.

$$e_i = y_i - \hat{y}_i$$

Visually, the residual is the vertical gap between each order's true value and the line's prediction.

Why does simply summing residuals fail

Naive idea: total error = $\Sigma_i e_i$. Problem: positive and negative residuals cancel. In an illustrative four-point toy example, two very different lines can both have a residual sum near zero — yet the poor line's individual errors are far larger. Summing signed residuals hides exactly the information we need.

Candidate error measures

Option A: $|e_i|$ (absolute error). Option B: e_i^2 (squared error).

Squared error is preferred here because it is differentiable everywhere (crucial for the optimization to come), and it penalizes large errors disproportionately more than small ones — a deliberate modeling choice.

The Squared-Error Objective

Sum of squared errors across all m orders:

$$\mathrm{SSE}(\beta) = \Sigma_i (\hat{y}_i - y_i)^2$$

SSE grows with the number of examples — awkward to compare across datasets of different sizes.

SSE, MSE, and Half-MSE: getting the names right

$$\mathrm{SSE} = \Sigma_i (\hat{y}_i - y_i)^2$$

$\mathrm{MSE} = (1/m)\, \Sigma_i (\hat{y}_i - y_i)^2$ — what mean_squared_error reports.

$J(\beta) = (1/(2m))\, \Sigma_i (\hat{y}_i - y_i)^2$ — half the MSE, not “the MSE.”

> **CAUTION · Key fact**
> SSE, MSE, and J differ only by positive multiplicative constants — they share the same minimizer β^*. “Ordinary Least Squares” means minimizing the sum of squares, independent of which constant multiple you write down. The ½ is included purely because it cancels neatly when we differentiate.

Learning as Optimization

The best parameters are those that minimize the loss:

$$\beta^* = \mathrm{argmin}_\beta\, J(\beta)$$

THINK ABOUT IT · Connect back

This is precisely the optimization problem structure from Lectures 13–14: an objective function, a search over a parameter space, a minimizer. Everything learned there about gradients and convexity applies directly here.

For least-squares linear regression, $J(\beta)$ as a function of (β_0, β_1) is a convex, bowl-shaped surface with a unique minimum — this is why the normal equation, derived in Act III, is guaranteed to find a single best answer.

The Fitted Line, Correctly Stated

Solving the optimization problem (mechanism derived in Act III) for Items ⟶ Order Value on the ShopKart data yields:

$$\beta_0 = 172.52 \qquad \beta_1 = 145.46$$

$$\hat{y} = 172.52 + 145.46 \cdot x$$

COMMON MISTAKE

$\beta_0 = 172.52$ is the intercept — the constant term, matching its position in $\hat{y} = \beta_0 + \beta_1 x$. $\beta_1 = 145.46$ is the slope — the coefficient multiplying x. Swapping which printed number is which is a mistake that compounds into wrong domain interpretation later.

Act II — Many Dimensions

From One Feature to Many

Order value probably depends on more than the item count — delivery distance, package weight, membership tier, and so on. The model generalizes directly:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_n x_{\text{in}}$$

Same recipe — prediction, residual, squared error, minimize. Nothing conceptually new, only more terms.

Vector Notation: One Example

Collect one order's features into a row vector, and prepend a constant 1 so the intercept can be absorbed into the same dot product:

$$x_i = [1, x_{i1}, x_{i2}, \ldots, x_{\text{in}}] \in \mathbb{R}^{n+1}, \quad \beta = [\beta_0; \beta_1; \ldots; \beta_n] \in \mathbb{R}^{n+1}$$

$$\hat{y}_i = x_i^T \beta$$

x_i is 1×(n+1), β is (n+1)×1, so $x_i^T\beta$ is a 1×1 scalar — exactly one prediction.

Vector Notation: The Whole Dataset

Stack all *m* orders' rows into a design matrix X; predictions for the entire dataset become one matrix-vector product.

$$X \in \mathbb{R}^{m\times(n+1)}, \quad y \in \mathbb{R}^m, \quad \hat{y} = X\beta \in \mathbb{R}^m$$

> **WORKED EXAMPLE · This is what 'vectorization' means**
>
> One matrix multiplication replaces m separate dot products — concise notation and fast code.

Planes and Hyperplanes

With one feature, $\hat{y} = \beta_0 + \beta_1 x$ is a line. With two features (Items, Delivery Distance), $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ is a plane. With more features, a hyperplane — not directly visualizable, but mathematically the same object.

Real fitted plane on the ShopKart data:

$$\hat{y} = 145.67 + 140.60 \cdot \text{Items} + 7.47 \cdot \text{Distance}$$

Act III — Solving It

Deriving the Normal Equation

Four short steps take us from the loss function to a closed-form solution for β. Every step is genuine algebra — nothing is asserted without derivation.

Part 1: Setting up

Write the loss in matrix form (up to a positive constant, the same objective as before):

$$J(\beta) = \|X\beta - y\|^2$$

Xβ−y is the vector of all m residuals at once; its squared (Euclidean) norm is exactly $\Sigma_i(\hat{y}_i - y_i)^2$, i.e., SSE.

Part 2: the gradient

The gradient of $J(\beta) = \|X\beta - y\|^2$ with respect to the vector β:

$$\nabla_\beta J(\beta) = 2X^T(X\beta - y)$$

Xβ−y is the residual vector (m×1); X^T ((n+1)×m) maps that residual back into "parameter space," giving a gradient of dimension (n+1)×1 — one number telling us how to nudge each β□.

Part 3: Setting the gradient to zero

At any minimizer β*, the gradient vanishes:

$$2X^T(X\beta - y) = 0 \quad \Longrightarrow \quad X^T X\beta = X^T y$$

A system of (n+1) linear equations — the normal equations.

Part 4: solving for β

If X^TX is invertible, multiply both sides by its inverse:

$$\beta^* = (X^T X)^{-1} X^T y$$

> **COMMON MISTAKE · Precondition**
> Valid only if X^TX is invertible — equivalently, the columns of X are linearly independent (no perfectly redundant features).

Singular or Ill-Conditioned: When the Normal Equation Fails

X^TX can fail to be invertible when features are duplicated or exactly linearly dependent — or nearly so, when features are highly correlated (e.g., Delivery Distance measured in km and an exact-copy column in miles).

> **KEY TAKEAWAY · The fix**
> The Moore–Penrose pseudoinverse X^+ generalizes matrix inversion and always exists, giving the least-squares solution of minimum norm even when X^TX is singular: $\beta^* = X^+y$.

Two Routes to the Same Destination

NOTE · Normal Equation

Exact, one-shot; scales poorly (roughly $O(n^3)$) with feature count; needs X^TX well-behaved.

CAUTION · Gradient Descent

Approximate, iterative; needs a learning rate and multiple passes; scales far better to large n.

$$\beta \leftarrow \beta - \alpha \cdot \nabla_\beta J(\beta) = \beta - \alpha \cdot (1/m)\, X^T (X\beta - y)$$

What scikit-learn Actually Does

The normal equation is the mathematical theory behind OLS. But LinearRegression does not explicitly form or invert X^TX in practice — that is, numerically less stable and often slower than necessary. It solves the least-squares problem with an SVD-based routine instead.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)          # solves least squares via SVD internally,
                                      # NOT by literally computing (XᵀX)⁻¹Xᵀy
```

Fitting the Model and Reading the Coefficients

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train[["ItemsInCart"]], y_train)

print(model.intercept_)   # β0 ≈ 172.52
print(model.coef_)        # [β1] ≈ [145.46]
model.predict([[10]])     # predicted order value (₹) for a 10-item cart
```

KEY TAKEAWAY · Interpretation (domain language)

"A slope of 145.46 means: in this simplified single-feature model, adding one more item to the cart is associated with an increase of about ₹145.46 in predicted order value, holding the (linear) relationship fixed." Units are stated explicitly — a bare number like '145.46' is meaningless without them.

Residual Diagnostics

Plot residuals against predicted values. A well-specified linear model should show residuals scattered randomly around zero, with no visible pattern.

```
import matplotlib.pyplot as plt

y_pred = model.predict(X_train[["ItemsInCart"]])
residuals = y_train - y_pred

plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Predicted Order Value"); plt.ylabel("Residual")
```

Act IV — When Lines Aren't Enough

Classroom Check: What If the Relationship Bends?

Consider delivery time as a function of delivery distance. A linear model drawn through this data would systematically miss the pattern at longer distances. What kind of function would fit this better — and could we still use linear regression to get it?

KEY TAKEAWAY · Yes — with a trick

Engineer a new feature (e.g., distance²) and fit a linear model on the expanded feature set. The model is still linear in its parameters.

NOTE · Why it bends

Simulated ShopKart relationship: delivery time grows faster than linearly with distance, due to increasing traffic and congestion on longer routes.

Polynomial Regression: The Model

Start with a single feature, extended by its square:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

Real fitted model on the ShopKart delivery-time data:

$$\hat{y} = 6.87 + 1.65 \cdot \text{Distance} + 0.095 \cdot \text{Distance}^2$$

The Key Conceptual Point: Linear in the Parameters

The curve is nonlinear in x (it curves) but linear in the parameters β (a weighted sum of terms, each multiplied by exactly one β□):

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 = [1, x, x^2] \cdot \beta$$

THINK ABOUT IT · Why this matters

This is precisely why polynomial regression is still a linear model, solvable by the exact same normal equation or gradient descent machinery already derived — only the design matrix X changes (it now has a column for x^2), not the solution method.

Interaction and Higher-Order Features

With multiple original features (Items x_1, Delivery Distance x_2), we can engineer new columns: x_1^2, x_2^2, and the interaction term x_1x_2 (capturing effects where the influence of one feature depends on the value of another):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_2^2 + \beta_5 x_1 x_2$$

PolynomialFeatures in scikit-learn

PolynomialFeatures(degree=d, include_bias=False) transforms n original features into every monomial of total degree ≤ d (excluding the constant) — powers of individual features and interaction terms.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_train[["ItemsInCart", "DeliveryDistance_km"]])
# X_poly columns: Items, Distance, Items², Items·Distance, Distance²
```

Exact output feature count (excluding the constant): C(n+d, d) − 1, the number of ways to choose a monomial of total degree ≤ d in n variables. The growth intuition matters more than memorizing the formula — it grows combinatorially with degree.

An End-to-End Pipeline

Chain the polynomial expansion and the linear model into a single Pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly_model = Pipeline([
    ("poly_features", PolynomialFeatures(degree=2, include_bias=False)),
    ("linear_reg", LinearRegression()),
])

poly_model.fit(X_train[["Distance_km"]], y_train)
poly_model.predict(X_test[["Distance_km"]])
```

WORKED EXAMPLE · Why a Pipeline

Notice that we never call .fit_transform() on the whole dataset before splitting — the Pipeline guarantees that the polynomial transform is only ever fit on the training data. A first-light touch on leakage, fully developed in Lecture 16.

Comparing Degrees: 1, 2, 6, and 15

Fitting the same delivery-time data with polynomial degrees 1, 2, 6, and 15 tells an increasingly dramatic story. The degree-15 curve passes almost exactly through every training point — its training error is nearly zero.

CAUTION

Is that a good thing? This question is deliberately left open here — it is the cliffhanger that opens Lecture 16: Why Regularization? Ridge Regression.

Conceptual Summary

WORKED EXAMPLE · Representation

$\hat{y} = X\beta$ — a (possibly feature-engineered) linear function of the parameters.

CAUTION · Evaluation

$J(\beta)$ — the squared-error loss — a scalar score for any candidate β.

KEY TAKEAWAY · Optimization

$\beta^* = \text{argmin}_\beta J(\beta)$ — solved exactly (normal equation) or iteratively (gradient descent).

THINK ABOUT IT · Complexity

Raw vs. engineered features: the feature representation determines what shapes a linear model can express.

Final Takeaway

KEY TAKEAWAY

“Linear regression is not ‘draw a line through points.’ It is: choose a parameterized function, define a prediction error, and solve an optimization problem to obtain the parameters.” Every model in the rest of this course — Ridge, Lasso, Logistic Regression, Softmax Regression — reuses this exact recipe with one or two pieces changed.

Closing

You Should Now Be Able to Explain…

- Why "the loss function" for OLS can be written in at least three equivalent forms (SSE, MSE, half-MSE), and why they all have the same minimizer.
- Why is polynomial regression classified as a linear model even though its predictions form a curve?
- What the normal equation says, where it comes from (via the gradient of J(β)), and under what conditions it can fail.
- Why scikit-learn's LinearRegression does not literally compute $(X^TX)^{-1}X^Ty$, and why that distinction matters numerically.
- How to read a fitted coefficient (model.coef_) in domain language, including its units.

Conceptual Check Questions

1. Two students fit the same data: one minimizes SSE, the other minimizes J(β) = (1/2m)·SSE. Will they obtain the same β*? Justify your answer without doing any calculus.
2. Suppose you accidentally include the same feature column twice in your design matrix X (e.g. ItemsInCart and an exact duplicate ItemsInCart_copy). What goes wrong when you try to apply the normal equation, and why?
3. A colleague says, "I used PolynomialFeatures(degree=10) and my training error dropped to almost zero — my model must be excellent." What is missing from this argument, and what would you check before agreeing?

Coding / Application Exercise

Using the ShopKart order dataset (or a similar tabular dataset you already have), build a Pipeline that (a) selects ItemsInCart as the sole feature, (b) fits an ordinary LinearRegression, and (c) reports intercept_, coef_, and the residual-vs-predicted plot. Then repeat with PolynomialFeatures(degree=3, include_bias=False) inserted before the regressor, and compare the two residual plots.

> **NOTE · Deliverable**
> One sentence stating, in domain language, what the degree-1 model's slope means, and one sentence describing what changed (visually) in the residual plot after moving to degree 3.

Concept Map: Connecting to DS605 So Far

- Optimization Unit (Lectures 13–14) — gradients, convexity, iterative solvers.
- Linear Regression (this lecture) — J(β), $\nabla_\beta J$, closed-form vs. iterative solving.
- Lecture 16: Regularization — next up.

Next Lecture

> **CAUTION · Lecture 16: Why Regularization? Ridge Regression**
>
> The high-degree curve fit the training data almost perfectly. Why might we reject it anyway?